

TVP_15_12_24@1

#technicke_vybaveni_pocitacu

Informace ve výpočetní technice

snímek 2

- informace obecně je nějaký údaj o dění v reálném světě, v informatice je to řetězec znaků, které lze vysílat, přijímat, uchovávat a zpracovávat

Číselné soustavy

snímek 3

- číselná soustava je způsob reprezentace čísel
- zápis čísla v dané soustavě je daná posloupností symbolů - číslic
- rozlišujeme spoustu soustav; mezi základní patří dvojková (binární), osmičková (oktální), desítková (decimální), šestnáctková (hexadecimální) nebo i šedesátková (u hodin)

Binární

snímek 4

- používá jen znaky nula a jedna reprezentující dva stavy: zapnuto/vypnuto, pravda/lež a tak dále.
- každá číslice odpovídá n -té mocnině čísla 2, kde n je pozice dané číslice v zapsaném čísle, začínajíc od nuly
- v AVRku značíme 0b (`napsat na tabuli`)

snímek 5

- pro převod z dvojkové soustavy do desítkové si jednotlivé číslice umocníme podle pozice číslice
 - pokud číslice je rovna 1 přičteme do celkového výsledku její poziční mocninu; pokud číslice je rovna 0 číslici přeskochíme
 - podle této logiky zjistíme, že `11010110` se rovná `214`

snímek 6

- a zde máme pravidla pro sčítání a odčítání
 - `na tabuli sečíst čísla 0111 + 1100 [=10011]`
 - `na tabuli odečíst čísla 101110 - 011100 [=100110]`

Terinární

snímek 7

- používá tři symboly: `0`, `1`, `2` nebo `-1`, `0`, `1`
- číslíci nazýváme trit

Decimální

snímek 8

- klasická desítková soustava umožňující zápis libovolných čísel

Hexadecimální

snímek 9

- má šestnáct číslic kde místo deset a dále máme písmena od A a jsem vám tu napsal i regulérní výraz
- v AVR uvádíme tuto soustavu znakem 0x (`napsat na tabuli`)

- šestnáctkovou soustavu používáme například v operační paměti pro adresy nebo když pracujeme s barvami (`#FFF`)
- z konstrukčního hlediska počítače pracují v soustavě dvojkové, v ní se mnohociferná čísla čtou špatně, proto je převádíme do šestnáctkové

BCD

snímek 10

- v této soustavě zapisujeme desítkové číslo nikoliv jako celek, ale jako jednotlivé číslice v binární soustavě
- tato informace se dá zapsat do jednoho nibblu, což je polovina bajtu nebo-li čtyřem bitům

Záporná čísla

snímek 11

- ve dvojkové soustavě používáme dvojkový doplněk
- bit nejvíce vlevo v registru určuje, zda číslo je kladný (při nule), či záporný (bit je jedna)
- pro znegování čísla invertujeme hodnotu bitu a přičteme 1 (`10110 [-01010]`)
- nevýhodou je asymetrický interval kdy je pozitivní číslo o jedno menší než záporný (`10110 [-01010]`)

$$\left\langle -\frac{2^n}{2}; \frac{2^n}{2} - 1 \right\rangle$$

Ochrana dat

snímek 12

- pokud dojde k přerušení linky a počítač přestane dostávat data, chyba se odhalí sama
- horší je, když počítač omylem přijme logickou nulu místo jedničky
- počítač chybu nerozpozná a bude dále pracovat se špatnými daty
- proto zavádíme ochranu dat

Paritní bit

snímek 13

- nejjednodušší způsob detekce chyb, jen a pouze detekce
- paritní bit přidáme k přenášenému řetězci jedniček a nul
- při sudé paritě máme v celkovém řetězci (což je původní i s bitem) sudý počet jedniček, u liché naopak lichý počet
- nastene-li ale chybe ve více bitech naráz, je nám paritní bit k ničemu; to samé i při chybném přijetí paritního bitu

CRC

snímek 14

- pomocí generujícího polynomu z přenášených dat se vypočte určitým způsobem zabezpečující kód, který následně připojíme na konec přenášeného řetězce
- výhodou je, že se kód může generovat během samotného přenosu
- po přenosu se mnohočlen v cíli znova vypočte a porovná se s přijatým
- je to metoda jednoduchá, ale efektivní

Endianita

snímek 15

- v jakém pořadí jsou v operační paměti uloženy jednotlivé řady čísel, který zabírají více, než jeden bajt
- při little-endian na paměťové místo s nejnižší adresou se uloží *nejméně významný bajt* a za něj se ukládají ostatní bajty až po *nejvíce významný bajt*

- u big-endian se na paměťové místo s nejnižší adresou se uloží *nejvíce významný bajt* a za něj se ukládají ostatní bajty až po *nejméně významný bajt*
- middle-endian je složitější způsob kombinující little a big-endian

Základní datové typy

Boolean

snímek 16

- v případech, kdy vlastnosti mohou mít jen dvě hodnoty
- spotřebují jen jeden bit v paměti k uložení stavu

Integer

snímek 17

- u většiny jazyků je číslo omezené intervalem a kódováno v dvojkovém doplňku
- pokud nepotřebujem záporná čísla, můžeme použít unsigned integer s větším maximálním číslem

Zlomek

snímek 18

- nulu v registru posuneme dle potřeby doleva (logické posuny fungují jako násobení/dělení)
- cokoliv za nulou bude $\frac{1}{2^n}$ kde n je pozice za nulou (indexujeme od **1**)

Znak

snímek 19

- ve skutečnosti je to integer u kterého každá hodnota představuje nějaký znak
- máme tabulky kódu UNICODE nebo ASCII a jejich národní rozšíření

Float

snímek 20

- je číslo s plovoucí desetinou čárkou
- na rozdíl od matematiky, kde tu v Česku oddělujeme desetinná místa čárkou, v západních zemích používají tečku, a to se využívá i v programování
- psáno v dvojkové soustavě, exponent je také celé číslo
- mnohá desetinná čísla nelze v tomto formátu přesně reprezentovat, proto se může stát, že se reálná čísla budou v počítači chovat jinak, než bychom čekali

String

snímek 21

- nějaký kus textu ohraničený uvozovkami, kde každý typ uvozovky může mít další funkce

Struktura procesorového systému

snímek 22

- datová cesta je přenosové médium informací propojující části procesoru
 - registry uchovávají dočasná data, instrukce nebo stavy
 - ALU provádí aritmetické a logické operace jako je AND, OR, XOR
 - a FPU zpracovává pohyblivou desetinnou čárku

snímek 23

- paměťová hierchie zajišťuje aby byl přístup k paměti nejrychlejší a nejefektivnější

snímek 24

- datová sběrnice zajišťuje přenos mezi procesorem a perifériemi
- napojuje se na datové cesty

snímek 25

- řídí pohyb dat z paměti, ALU, externí periférie; načítá instrukce a celkově vše řídí

snímek 26

- vstupy a výstupy prováděné řadičem a sběrnici
- paralelní zpracování dat
 - vícero jádry nezávisle
 - v pipeline jsou instrukce zpracovány v různých fázích současně

Program jako data

snímek 27

- co píšeme programy, vlastně jen píšeme textové soubory obsahující instrukce
 - s tímto souborem můžeme nakládat jako s jakýmkoliv jiným
- abychom mohli vykonat instrukce ze souboru musíme použít jiný program k přeložení do kódu strojového
 - kompilátor instrukce přeloží a zabalí je do spustitelného balíčku (například .exe)
 - interpreter instrukce překládá zapochodu, nikoliv do exe

snímek 28

- to je vše